CSC315 Number System, Codes and Addressing Modes (2)

Condensed Study Notes
Generated: 2026-05-24 23:08 UTC

COMPUTER ARCHITECTURE AND ORGANIZATION AFIT, KADUNA LECTURE NOTE

Welcome to the exciting world of how computers work from the inside out!

Imagine a computer is like a very organized office. This lecture note is going to guide us through understanding that organization.

We'll be looking at two main things:

1. **Computer Architecture**: This is like the blueprint of the office. It describes how the different parts of the office (like the desks, filing cabinets, and people) are arranged and how they relate to each other to get work done.

2. **Computer Organization**: This is more about how the actual work happens in the office. It's about the specific devices and the connections between them that allow the tasks to be performed efficiently. Think of it as the day-to-day operations and the tools used.

By understanding these two aspects, we can start to see how a computer takes instructions and data and turns them into useful results.

1.0 Number System

We've talked about Computer Architecture as the blueprint and Computer Organization as the operational details. Now, to understand how computers actually work with information, we need to talk about how they represent numbers.

Imagine you have different ways to count things. You could use your fingers (which is like a system with 10 digits), or maybe you only have two special tokens (like 'on' and 'off') to represent everything.

Different ways of counting or representing numbers are called 'number systems'. Each number system has a 'base' (also called 'radix'). The base tells us how many unique digits or symbols are available in that system.

Think of the base like the number of fingers you have. If you have 10 fingers, you can easily count up to 10. If you only had 2 special tokens, you'd have to get creative to represent more than just 'yes' or 'no'.

The rule is: if the base of a number system is 'r', then the numbers in that system will range from zero up to 'r-1'. The total count of unique numbers you can use is exactly 'r'.

By picking different values for this base (as long as it's two or more), we can create different number systems. The most common ones we'll encounter are:

• **Decimal Number system**: This is the system we use every day! It has a base of 10, meaning it uses 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9.

• **Binary Number system**: This system is super important for computers. It has a base of 2, meaning it only uses two digits: 0 and 1. Think of it like a light switch – it's either 'off' (0) or 'on' (1).

• **Octal Number system**: This system has a base of 8, using digits from 0 to 7.

• **Hexadecimal Number system**: This system has a base of 16. It uses digits 0-9 and then letters A-F to represent values beyond 9.

1.1 Decimal Number System

We're all used to counting and working with numbers every day, and the system we use is called the decimal system. Think of it like having a special way to organize your LEGO bricks. You have different bins for different sizes, and the number of bricks in each bin tells you how many of that size you have.

The decimal system uses ten different digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. This is why it's called the 'decimal' system – 'deci' relates to ten. It's probably no coincidence that humans have ten fingers and ten toes, which might be why this system became so common!

This system is a **positional number system**. This means that the position of a digit within a number tells you its value. Imagine a row of boxes, and each box represents a different place value. The further to the left a box is, the more it's worth.

Each position in the decimal system has a specific 'weight' or value. This weight is a power of 10. The position on the far right usually represents 10 to the power of 0 (which is 1), the next position to the left represents 10 to the power of 1 (which is 10), the next is 10 to the power of 2 (which is 100), and so on.

Let's take the number 4728 as an example.

- The '8' is in the ones place (10^0).
- The '2' is in the tens place (10^1).
- The '7' is in the hundreds place (10^2).
- The '4' is in the thousands place (10^3).

So, 4728 really means (4 *1000)* + *(7* 100) + (2 *10)* + *(8* 1) = 4000 + 700 + 20 + 8.

This same idea applies to numbers with decimal points (fractions). For these, we use negative powers of 10. The position just to the right of the decimal point is the tenths place (10^-1 or 1/10), the next is the hundredths place (10^-2 or 1/100), and so on.

For example, the decimal 0.256 means:

- The '2' is in the tenths place (10^-1).
- The '5' is in the hundredths place (10^-2).
- The '6' is in the thousandths place (10^-3).

So, 0.256 is equal to (2 *1/10)* + *(5* 1/100) + (6 * 1/1000) = 0.2 + 0.05 + 0.006.

1.1.1 The Binary System

You know how we use a number system with ten digits (0-9) every day? Well, computers don't use that! They use a different number system called the **binary system**. This is the system that all digital circuits and computer systems rely on.

The binary system has a **base** (or **radix**) of just 2. This means it only uses two digits: 0 and 1.

Think of it like a light switch. It can only be ON (represented by 1) or OFF (represented by 0). There's no in-between.

Just like our decimal system has a decimal point, the binary system has a **binary point**. Things to the left of the binary point are the 'whole number' part, and things to the right are the 'fractional' part.

Each position in a binary number has a specific 'weight', which is a power of 2.

- To the left of the binary point:
- The first position is worth 2 to the power of 0 (2^0), which is 1.
- The next position is worth 2 to the power of 1 (2^1), which is 2.
- The next is 2 to the power of 2 (2^2), which is 4, and so on (2^3, 2^4, etc.).
- To the right of the binary point:
- The first position is worth 2 to the power of -1 (2^-1), which is 1/2.
- The next is 2 to the power of -2 (2^-2), which is 1/4, and so on (2^-3, 2^-4, etc.).

Example 1

Let's see how we can convert a binary number (which uses only 0s and 1s) into a decimal number (the one we use every day). We do this by looking at each digit in the binary number and giving it a 'weight' based on its position.

Think of it like a special kind of odometer in a car. Each digit on the odometer has a value, and its position determines how much it contributes to the total mileage. For binary numbers, these positions have specific mathematical values.

Consider the binary number **1101.011**.

This number has two parts: an integer part (**1101**) and a fractional part (**0.011**).

For the integer part (**1101**), each digit has a weight based on powers of 2, starting from 2^0 for the rightmost digit and increasing as we move left:

- The '1' in the 2^3 position (which is 8) has a weight of 2^3.
- The '1' in the 2^2 position (which is 4) has a weight of 2^2.
- The '0' in the 2^1 position (which is 2) has a weight of 2^1.
- The '1' in the 2^0 position (which is 1) has a weight of 2^0.

For the fractional part (**0.011**), the weights are based on negative powers of 2, starting from 2^-1 for the digit immediately after the decimal point and decreasing as we move right:

- The '0' in the 2^-1 position (which is 1/2 or 0.5) has a weight of 2^-1.
- The '1' in the 2^-2 position (which is 1/4 or 0.25) has a weight of 2^-2.
- The '1' in the 2^-3 position (which is 1/8 or 0.125) has a weight of 2^-3.

By summing up the values of each digit multiplied by its corresponding weight, we can find the equivalent decimal number. This process simplifies the binary representation into its decimal form.

a) Decimal Number to other Bases Conversion

We know that our everyday numbers are in the 'decimal' system (base-10). Now, let's learn how to convert these decimal numbers into other number systems, like binary (base-2), octal (base-8), or hexadecimal (base-16).

Imagine you have a number with a whole part (like 123) and a part after the decimal point (like .45). We need to handle these two parts separately when converting.

Converting the Whole Part (Integer Part):

Think of this like trying to fit a large number of items into smaller boxes. You keep dividing the items and seeing how many full boxes you get and how many are left over.

1. **Divide:** Take the whole part of your decimal number and keep dividing it by the 'base' (the number system you're converting to, like 2 for binary).

2. **Note the Remainder:** Each time you divide, write down the 'remainder' (what's left over after dividing as evenly as possible).

3. **Repeat:** Use the result of the division (the 'quotient') as your new number and divide it again by the base.

4. **Keep Going:** Continue this process until the quotient becomes zero.

5. **Read Backwards:** The remainders you wrote down, when read from bottom to top (the last remainder first), give you the equivalent number in the new base.

- The very first remainder you got is the 'least significant digit' (the rightmost digit) in the new base.
- The very last remainder you got is the 'most significant digit' (the leftmost digit) in the new base.

Converting the Fractional Part (Part after the Decimal Point):

This is a bit like trying to measure out precise amounts of liquid. You keep taking a portion and seeing how much you have left.

1. **Multiply:** Take the fractional part of your decimal number and multiply it by the 'base' (the number system you're converting to).

2. **Note the Carry:** Look at the whole number part of the result. This is your 'carry'. Write it down.

3. **Take the New Fraction:** Use only the fractional part of the result from step 1 and multiply it by the base again.

4. **Repeat:** Continue this process. You'll get a sequence of carries.

5. **Read Forwards:** The carries you've collected, when read in the order you got them (from top to bottom), form the fractional part of the equivalent number in the new base.

6. **When to Stop:** You stop when the fractional part becomes zero, or when you have enough digits to match the precision you need.

1.1.2 CONVERTING BETWEEN BINARY AND DECIMAL

We've learned that computers speak in 'binary' (using only 0s and 1s) and we usually use 'decimal' (our everyday numbers with 0-9). Now, let's see how to switch between these two.

From Binary to Decimal: Like Counting with Different Tools

Imagine you have a special set of coins, where each coin's value is double the one before it (like 1 cent, 2 cents, 4 cents, 8 cents, and so on). If you want to make a specific amount, say 13 cents, you'd pick the coins that add up to it. For 13 cents, you might use an 8-cent coin, a 4-cent coin, and a 1-cent coin (8 + 4 + 1 = 13).

Binary works the same way! Each digit in a binary number is like a position for these special coins. From right to left, the positions represent powers of 2: 2^0 (which is 1), 2^1 (which is 2), 2^2 (which is 4), 2^3 (which is 8), and so on.

To convert a binary number to decimal:

- **Assign values:** Write down the binary number and, below each digit, write its corresponding power of 2 (starting from 2^0 on the right).
- **Multiply and sum:** For each digit in the binary number, multiply the digit (0 or 1) by its assigned power of 2.
- **Add them up:** Sum all the results from the multiplication step. This total is the decimal equivalent.

For example, the binary number `1101`:

`1` *(at the 2^3 position) = 1* 8 = 8

`1` *(at the 2^2 position) = 1* 4 = 4

`0` *(at the 2^1 position) = 0* 2 = 0

`1` *(at the 2^0 position) = 1* 1 = 1

Adding these up: 8 + 4 + 0 + 1 = 13. So, `1101` in binary is `13` in decimal.

From Decimal to Binary: Like Making Change with Specific Bills

Think about giving someone change. If they need to pay $13, and you only have $8, $4, and $1 bills, you'd first give them the biggest bill that's not more than $13 (the $8 bill). You'd then have $5 left ($13 - $8). Next, you'd give them the biggest bill not more than $5 (the $4 bill), leaving you with $1 ($5 - $4). Finally, you give them the $1 bill. You used an $8, a $4, and a $1 bill.

Converting decimal to binary involves a similar process of figuring out which 'powers of 2' (our special coins) are needed.

To convert a decimal number to binary:

- **Find the largest power of 2:** Find the largest power of 2 that is less than or equal to your decimal number.
- **Subtract and record:** Subtract this power of 2 from your decimal number. Put a '1' in the corresponding position in your binary number.
- **Repeat:** Take the remaining number and repeat the process, finding the largest power of 2 less than or equal to it. If a power of 2 can't be used (because it's larger than the remaining number), put a '0' in that position.
- **Continue until zero:** Keep going until your remaining number is zero.

For example, to convert the decimal number 13 to binary:

- The largest power of 2 less than or equal to 13 is 8 (which is 2^3). So, we have a '1' in the 2^3 position. Remaining: 13 - 8 = 5.
- The largest power of 2 less than or equal to 5 is 4 (which is 2^2). So, we have a '1' in the 2^2 position. Remaining: 5 - 4 = 1.
- The largest power of 2 less than or equal to 1 is 1 (which is 2^0). So, we have a '1' in the 2^0 position. Remaining: 1 - 1 = 0.
- We didn't use 2^1 (which is 2), so we put a '0' in that position.

Putting it together, we get `1101` in binary (the '1's correspond to the powers of 2 we used: 2^3, 2^2, and 2^0).

Example 2

Let's walk through an example of converting a decimal number that has both a whole part and a fractional part into its binary equivalent. We'll use the decimal number 58.25.

Imagine you have a number like 58.25, where 58 is the whole number part and 0.25 is the part after the decimal point (the fractional part).

To convert the whole number part (58) to binary, we use a process of repeated division by 2, keeping track of the remainders. Think of it like trying to make change for 58 cents using only 2-cent coins, and noting down if you have any cents left over each time.

- **58 divided by 2** gives 29 with a remainder of **0**. This remainder is the first binary digit (from the right).
- **29 divided by 2** gives 14 with a remainder of **1**.
- **14 divided by 2** gives 7 with a remainder of **0**.
- **7 divided by 2** gives 3 with a remainder of **1**.
- **3 divided by 2** gives 1 with a remainder of **1**.
- **1 divided by 2** gives 0 with a remainder of **1**. This is the last binary digit (on the left).

Reading the remainders from bottom to top, the whole number part 58 in decimal is **111010** in binary.

Now, let's convert the fractional part (0.25) to binary. For this, we use repeated multiplication by 2 and keep track of the 'carries' (the whole number part of the result). Imagine you have 0.25 of a dollar and you want to see how many 50-cent (0.5) pieces you can make, and what's left over.

- **0.25 multiplied by 2** gives **0.5**. The whole number part (the carry) is **0**. This is our first binary digit after the decimal point.
- We take the fractional part of the result (0.5) and multiply it by 2: **0.5 multiplied by 2** gives **1.0**. The whole number part (the carry) is **1**. This is our next binary digit.

Since the result is exactly 1.0, we stop. The fractional part 0.25 in decimal is **.01** in binary.

Putting it all together, the decimal number 58.25 is **111010.01** in binary.

b) Decimal to Binary Conversion

Now, let's figure out how to change a number we use every day (a decimal number) into the kind of number a computer understands (a binary number).

It's like converting a price in dollars to its equivalent in a different currency. We need a systematic way to do this.

When we convert a decimal number to binary, we perform two main types of operations, depending on whether we're dealing with the whole part or the fractional part of the number:

1. **For the whole number part:** We repeatedly divide the whole number by 2. We keep track of the remainders. Think of this like making change for a large bill using only smaller bills – you keep taking out the largest possible denomination (in this case, groups of 2) and note down what's left over.

2. **For the fractional part (the part after the decimal point):** We repeatedly multiply the fractional part by 2. We then take the whole number part of the result. This is similar to trying to get a precise measurement in a different unit – you keep multiplying and taking the whole unit you get until you've accounted for the whole original fraction.

These two operations, division for the whole part and multiplication for the fractional part, are the core steps to convert a decimal number into its binary form.

Example 3

Let's look at an example of converting a binary number to its decimal equivalent. We'll take the binary number 1101.11.

Remember how we learned that each digit in a binary number has a specific 'place value' based on powers of 2? For numbers with a decimal point (or a 'binary point' in this case), the places to the right of the point represent fractions using negative powers of 2.

So, for 1101.11:

- The '1' in the leftmost position is in the 2^3 (which is 8) place.

- The next '1' is in the 2^2 (which is 4) place.
- The '0' is in the 2^1 (which is 2) place.
- The '1' just before the point is in the 2^0 (which is 1) place.
- The first '1' after the point is in the 2^-1 (which is 0.5) place.
- The second '1' after the point is in the 2^-2 (which is 0.25) place.

To find the decimal equivalent, we multiply each binary digit by its corresponding place value and add them all up:

(1 *8)* + *(1* 4) + (0 *2)* + *(1* 1) + (1 *0.5)* + *(1* 0.25)

This gives us:

8 + 4 + 0 + 1 + 0.5 + 0.25 = 13.75

So, the binary number 1101.11 is equal to 13.75 in the decimal system we use every day.

c) Binary Number to other Bases Conversion

We've learned how to turn binary numbers (the computer's language of 0s and 1s) into decimal numbers (our everyday language). Now, let's look at how to convert binary numbers into other number systems, like octal (base-8) or hexadecimal (base-16).

It's important to know that converting from binary to decimal uses a different method than converting from binary to these other bases. Think of it like learning two different ways to count: one way to count apples (decimal) and another way to count pairs of socks (binary to octal/hexadecimal).

This section will focus on the specific techniques for converting binary numbers into different number bases. We'll explore how to group binary digits to make these conversions easier.

d) Binary to Hexa-Decimal Conversion

We've learned how computers use binary (base-2, with just 0s and 1s), but sometimes it's easier for us to read numbers in other systems. We're now going to look at converting from binary to hexadecimal.

Think of hexadecimal as a shorthand for binary. Imagine you have a long string of binary digits, like a very long phone number. It can be hard to read and remember. Hexadecimal is like grouping those digits into shorter, more manageable chunks. This works because there's a neat relationship between binary and hexadecimal.

The binary system has a 'base' of 2 (meaning it uses 2 digits: 0 and 1). The hexadecimal system has a 'base' of 16 (meaning it uses 16 digits: 0-9 and then A-F).

Here's the cool part: four binary digits (called 'bits') can represent exactly the same number as one hexadecimal digit. This is because 2 raised to the power of 4 (2■) equals 16 (2 *2* 2 * 2 = 16).

To convert a binary number to hexadecimal, we follow these steps:

1. **Grouping the bits:** Starting from the 'binary point' (which is like the decimal point but for binary numbers), group the binary digits into sets of four. You'll do this for the digits on both sides of the binary point.

    • If a group doesn't have exactly four bits, you can add zeros to the 'extreme sides' (the furthest left for the whole part, and the furthest right for the fractional part) to complete the group of four. Think of these added zeros as padding to make the groups uniform.

2. **Converting each group:** Once you have your groups of four bits, you'll replace each group with its corresponding hexadecimal digit.

    • For example, the binary group '1010' would become the hexadecimal digit 'A'. We'll see how these specific conversions work in practice in future steps.

Binary to Decimal Conversion

We know that computers speak in binary (using only 0s and 1s), but we humans usually prefer to understand numbers in decimal (0-9). So, we need a way to translate between these two.

Think of it like having a special set of coins for each position in a binary number. Each coin's value depends on its position, just like how the '1' in 100 is worth more than the '1' in 10.

To convert a binary number to its decimal equivalent, we do two main things:

1. **Assign 'weights'**: Each digit (called a 'bit') in the binary number gets a specific value based on its place. This 'weight' is determined by powers of 2, starting from the rightmost bit.

2. **Multiply and Add**: We multiply each binary bit (either 0 or 1) by its assigned positional weight. Then, we add up all these results to get the final decimal number.

Example 4

Let's look at how to turn a binary number (the computer's language of 0s and 1s) into a hexadecimal number (a shorter way to write those 0s and 1s). Think of it like using abbreviations for long phrases. Instead of saying "the quick brown fox jumps over the lazy dog" every time, we might just say "Tqbfjotld".

Here's how we convert the binary number `101110.01101` to hexadecimal:

• **Step 1: Grouping the bits.** We take the binary number and group its digits (called **bits** when they are 0 or 1) into sets of four, starting from the **binary point** (like a decimal point, but for binary numbers). We group them moving away from the binary point in both directions.

• For the whole number part (`101110`), we group from the right: `10 1110`.

• For the fractional part (`01101`), we group from the left: `0110 1`.

• **Step 2: Making full groups of four.** If a group doesn't have four bits, we add zeros to the outside to make it a full group of four.

• For `10 1110`, the first group `10` only has two bits. We add two zeros to the left to make it `0010`.

• For `0110 1`, the last group `1` only has one bit. We add three zeros to the right to make it `1000`.

• So, our binary number now looks like: `0010 1110.0110 1000`.

• **Step 3: Converting each group to a hexadecimal digit.** Each group of four binary bits now directly corresponds to a single hexadecimal digit. The original binary number `101110.01101` becomes `0010 1110.0110 1000`.

• `0010` in binary is `2` in hexadecimal.

• `1110` in binary is `E` in hexadecimal.

• `0110` in binary is `6` in hexadecimal.

• `1000` in binary is `8` in hexadecimal.

• **Result:** Putting these together, the binary number `101110.01101` is equivalent to the hexadecimal number `2E.68`.

Conversely, converting from hexadecimal to binary is like expanding those abbreviations. You take each hexadecimal digit and replace it with its 4-bit binary equivalent.

Example 5

Let's look at an example of converting a number from the hexadecimal system to the binary system.

Imagine you have a secret code where each letter or number can be represented by a small group of symbols. The hexadecimal system is a bit like that, but instead of symbols, we use digits (0-9) and letters (A-F). It's a way to write down long binary numbers (which computers use) in a shorter, more human-friendly way.

We know that each digit in the hexadecimal system can be neatly represented by exactly four digits in the binary system (which uses only 0s and 1s). This is because four binary digits can represent 16 different combinations, and there are exactly 16 unique characters in the hexadecimal system (0-9 and A-F).

Let's take the hexadecimal number `65.4C`.

To convert it to binary, we look at each hexadecimal digit and replace it with its 4-bit binary equivalent:

• `6` in hexadecimal becomes `0110` in binary.

• `5` in hexadecimal becomes `0101` in binary.

• `.` is the decimal point, which stays the same.

• `4` in hexadecimal becomes `0100` in binary.

• `C` in hexadecimal becomes `1100` in binary.

Putting these together, we get `01100101.01001100`.

Now, a cool thing about binary (and numbers in general) is that leading zeros (zeros at the very beginning of a number) and trailing zeros (zeros at the very end of a number) don't change its actual value. It's like writing `007` instead of `7` – it's still seven.

So, when we look at `01100101.01001100`:

• The leading `0` before the `0110` (the first group) can be removed, giving us `1100101`.

• The trailing `0` at the end of `01001100` can be removed, giving us `010011`.

Therefore, the hexadecimal number `65.4C` is equivalent to the binary number `1100101.010011`.

Key takeaway:

- Hexadecimal is a shorthand for binary.
- Each hexadecimal digit corresponds to a group of four binary digits (bits).
- Leading and trailing zeros in binary can be removed without changing the number's value.

2.1 Instruction Codes

Imagine you're giving instructions to a robot. You need to tell it not just *what* to do (like 'pick up the box'), but also *where* to find the box and *where* to put it once it's picked up.

In a computer, an **instruction code** is like a very specific command for the computer's central processing unit (CPU). It's a sequence of bits (those 0s and 1s we've talked about) that tells the computer exactly what needs to happen.

These instructions are performed on data that's stored in special temporary storage areas called **registers**. So, an instruction code needs to specify:

- **The operation to perform:** What action should the computer take? This is like telling the robot to 'pick up' or 'put down'. This part of the instruction is called the **operation code** (or **opcode**).

Where to find the data (operands): *Which registers hold the information the operation needs? This is like telling the robot* which* box to pick up.

Where to store the result: *Which register should the outcome of the operation be saved in? This is like telling the robot* where* to put the box after picking it up.

The **operation code** itself is a part of the instruction code made up of bits. It defines the specific action, like adding numbers, subtracting them, multiplying, shifting bits around, or complementing (flipping 0s to 1s and vice-versa). The number of different operations a computer can perform determines how many bits are needed for the operation code. For example, if a computer can do 8 different operations, it needs at least 3 bits for the operation code, because 2^3 = 8.

A **program** is simply a collection of these instruction codes, arranged in a specific order to achieve a larger task. Each instruction code tells the computer a tiny part of the overall job.

2.0 Arithmetic Logical Unit

Imagine a computer's processing unit as a busy workshop. Instead of having many small workbenches (registers) doing individual tasks, there's a central, super-powered workbench called the Arithmetic Logical Unit (ALU).

The ALU is like the main engine or the 'calculator' of the computer's brain (CPU). It's where all the number crunching and decision-making happens.

Here's how it works:

- **Getting Ready:** Data from specific storage spots (registers) is brought to the ALU's input.

• **Doing the Work:** The ALU performs the requested task. This could be a math calculation (like adding two numbers) or a logical comparison (like checking if one number is bigger than another).

• **Putting it Away:** Once the operation is finished, the result is sent to another designated storage spot (destination register).

2.2.1 Common Bus System

Imagine a busy office where people need to share information. Instead of each person having a direct line to everyone else (which would be a mess of wires!), they use a central hallway or a shared notice board to pass messages around. This is similar to how a 'common bus system' works in a computer.

A common bus system is like a shared pathway or a set of wires that allows different parts of the computer to send and receive data. Instead of having dedicated connections between every single component, data can travel along this common path.

In a basic computer, there are several key parts:

• **Registers**: These are like small, super-fast storage spots inside the computer's main processing unit (which we learned about earlier, containing the ALU). They temporarily hold data that the processor is currently working with.

• **Memory Unit**: This is where the computer stores larger amounts of data and instructions for longer periods.

• **Control Unit**: This is like the traffic director, managing and coordinating all the operations within the computer.

To move data from one register to another, or between a register and the memory, we need pathways. A common bus system provides an efficient way to do this.

Here's how it works:

• The outputs (where data comes out) of the registers and the memory unit are all connected to this common bus.

• This means any of these components can put their data onto the bus, and other components can then read it from the bus.

• It's a shared resource, so only one component can typically send data onto the bus at any given time to avoid confusion.

2.2 Stored Program Organisation

Imagine your computer's memory like a library. In this library, there are two main sections: one for the 'instruction manuals' (the steps the computer needs to follow) and another for the 'ingredients' (the actual information or data the computer will work with).

The simplest way to set up a computer is to have a special holding spot for information, called a **Processor Register**. In this setup, the computer's commands, called **instruction codes**, are made of two parts:

The Operation: *This part tells the computer* what* to do (like add, subtract, or move data).

The Address: *This part tells the computer* where* to find the information (the **operand**) it needs to perform the operation. This address points to a specific location in the memory.

Some computers use a single, primary Processor Register. This special register is called an **Accumulator (ACC)**. When the computer needs to do a calculation or a logical step, it uses the information stored in the Accumulator and the information it finds at the specified memory address.

Load (LD

Before we look at how instructions are structured, let's first understand how computers figure out where to find the information (or 'operands') they need to work with.

Think of the computer's 'common bus' like a shared highway that all the different parts of the computer use to send and receive information. When a register (a small, super-fast storage spot inside the computer) is told to 'load' (LD) something, it means it's ready to receive data from this highway.

Imagine you're telling a cashier what you want to buy. The way you tell them can be different:

Immediate Operand: *Sometimes, the instruction itself directly tells you the value you need. It's like saying, "I want to buy* three* apples." The number 'three' is right there in the instruction.

Direct Address: *Other times, the instruction tells you* where* to find the value. It's like saying, "Go to aisle 5 and get the apples." The instruction gives you the location (address) of the operand.

Indirect Address: *This is like a treasure hunt! The instruction tells you to go to a specific spot (a memory word) where you'll find another clue, which is the actual address of the operand. It's like saying, "Go to the information desk, and they'll tell you where the apples are." So, the instruction points to a location that* holds* the address of the operand.

2. Register Reference Instruction

Imagine you have a special remote control for a very specific appliance, like a smart coffee maker. This remote has different buttons, and some buttons are dedicated to controlling just the coffee maker's internal functions, not making coffee itself.

In a computer, there are also special types of instructions that are designed to control the internal workings of the computer's components, rather than performing calculations or moving data around in the usual way. These are called **Register Reference Instructions**.

Here's how they work:

- **Identifying the instruction:** These instructions are like a secret code. They start with a specific pattern of bits (which are like the 0s and 1s we've talked about) called an **opcode** (short for operation code). For these specific instructions, the opcode is '111'.
- **A special signal:** To make sure it's a Register Reference Instruction and not something else, there's an extra check. The very first bit (the leftmost one) must be a '0'. So, the complete 'address' for this type of instruction looks like '0111' followed by other bits.

What they do: *The remaining bits in the instruction (12 of them in this case) tell the computer exactly* which* internal operation to perform. These operations are focused on managing or manipulating the computer's internal parts, like its memory or processing units, rather than direct data work.

1. Memory Reference Instruction

Imagine you're sending a package, and you need to tell the postal service exactly where to find it. A Memory Reference Instruction is like a special set of instructions for the computer that tells it where to find information (data) it needs in its 'storage room' (memory).

This instruction uses a total of 13 bits (which are like tiny on/off switches, representing 0s and 1s).

- 12 of these bits are used to specify the exact 'address' or location of the data in the computer's memory. Think of this like the street number and house number for your package.
- The remaining 1 bit is used to specify the 'addressing mode'. This is like telling the postal service *how* to use the address you gave them.
- If this bit is a '0', it means the address you provided is the *direct* location of the data. It's like saying, "The package is right at this house number."
- If this bit is a '1', it means the address you provided is actually the location of *another address* where the data is stored. This is called an 'indirect address'. It's like saying, "Go to this house number, and at that house, you'll find another note telling you the real address of the package."

3. Input-Output Instruction

Imagine you have a special remote control for your computer that can tell it to do things like get information from a keyboard or send information to a screen. These are called Input-Output (I/O) instructions.

- **Operation Code**: Think of this like a secret code word that tells the computer what kind of task to perform. For these I/O instructions, the code word is '111'.
- **Leftmost Bit**: This is like a special flag or a specific part of the code word that signals 'this is an I/O instruction'. In this case, if the very first bit (the one furthest to the left) is a '1', it confirms it's an I/O instruction.
- **Remaining Bits**: After the code word and the flag, there are 12 more bits. These bits are like specific details or sub-commands that tell the computer *exactly* what input or output operation to do. For example, they might specify 'read from the keyboard' or 'write to the display'.

2.2.2 Computer Instructions

Think of computer instructions like a very specific recipe for a robot.

Each instruction tells the robot exactly what to do, what ingredients (data) to use, and where to put the finished product. The basic computer we're looking at uses instructions that have a specific structure, like a form with different boxes to fill in.

These instructions are divided into two main parts:

Operation Code (opcode): *This is like the main verb in a command, telling the computer* what* action to perform (e.g., 'add', 'move', 'compare'). In this basic computer, the opcode uses 3 bits to define the operation.

Remaining Bits: *After the opcode, there are 13 other bits. What these bits mean depends entirely on the specific operation code that was used. They might tell the computer* where *to find the data it needs or* where* to put the result.

a) Immediate Mode

Imagine you're telling someone to do something, and you give them the exact thing to use right away, without them having to go find it somewhere else. That's the idea behind Immediate Mode.

In computer instructions, Immediate Mode means the instruction itself contains the actual piece of data (called an **operand**) that the computer needs to perform its task. So, instead of telling the computer where to *find* the data, you're just giving it the data directly within the command.

For example, an instruction like 'ADD 7' tells the computer to add the number 7 to whatever is currently stored in a special place called the **accumulator** (which we learned is a register used for calculations). Here, '7' is the operand, and it's given immediately within the instruction itself.

Key points:

- The instruction directly includes the data (operand) it needs.
- There's no need for the computer to look up the data elsewhere (like in memory).
- The instruction has a field for the operand instead of a field for an address.

Format of Instruction

Imagine an instruction is like a message sent to the computer telling it what to do. This message is broken down into different parts, like a letter with a specific purpose, a recipient's address, and a special instruction on how to find something.

These parts are represented visually as a rectangular box, showing all the bits (those 0s and 1s we talked about) that make up the instruction.

Here are the main pieces of an instruction:

1. **Operation Code (Opcode) Field**: This is like the main verb of the instruction. It tells the computer *what* action to perform. For example, it might say 'add', 'subtract', or 'move data'.

2. **Address Field**: This part is like the street address for information. It tells the computer *where* to find the data (called an **operand**) it needs to perform the operation. This could be a specific spot in the computer's **memory** (a storage area) or a **register** (a small, super-fast storage location within the CPU itself).

3. **Mode Field**: This is like a hint or a special instruction on *how* to find the actual data. It specifies the method for figuring out the 'effective address' – which is the final, precise location of the operand. For instance, it might tell the computer to look directly at the address provided, or to follow a chain of addresses to find the data.

Computers can have instructions that are different lengths, meaning they have varying numbers of these address fields. The exact organization of these fields often depends on how the computer's internal registers are set up.

2.2.3 Addressing Modes and Instruction Cycle

Imagine you're giving instructions to someone, like a chef. The 'operation field' of a computer instruction is like telling the chef *what* to do – 'chop', 'stir', 'bake'. This action will be performed on some 'data', which is like the ingredients. These ingredients can be found either in the computer's 'registers' (small, super-fast storage spots right next to the chef) or in the 'main memory' (a larger pantry further away).

The 'addressing mode' of an instruction tells the computer *how* to find those ingredients (the data). It's like giving the chef different ways to get the ingredients: maybe the ingredient is right in front of them, or maybe they have to go to a specific shelf in the pantry.

Why do we have these different ways to find data? It's like having different tools for different jobs:

- **Programming Versatility:** It gives you more flexibility as a programmer, allowing you to organize your data and instructions in smarter ways. Think of it like a chef having multiple ways to get ingredients, which can speed up cooking.
- **Fewer Bits:** It helps save space in the instruction itself. By using clever addressing modes, we can use fewer bits (those 0s and 1s we talked about) to tell the computer where to find the data, making instructions more compact.

Types of Addressing Modes

We've learned that computer instructions need to know where to find the data they work with. This is where 'addressing modes' come in – they are like different ways of giving directions to find that data.

Think of it like finding a book in a library:

- **Directly telling you the shelf number:** This is like having the exact location.
- **Telling you which catalog card to look at, and that card has the shelf number:** This is like an extra step to find the location.
- **Giving you a clue, and that clue leads you to another clue, and eventually to the shelf:** This is like a trail of breadcrumbs.

These different ways of finding data help make instructions flexible and efficient. We'll look at these specific ways, one by one, to understand how the computer knows exactly where to get the information it needs for each instruction.

b) Register Mode

Imagine the CPU (the computer's brain) has a few small, super-fast scratchpads right inside it where it can quickly jot down numbers or pieces of information it's currently working with. These scratchpads are called **registers**.

In Register Mode, when the computer needs a piece of information (called an **operand**) to do a job, the instruction tells it which one of these internal scratchpads (registers) the information is already sitting in. It's like the instruction saying, 'Go look at scratchpad number 3 for the number you need.'

So, instead of the instruction needing to tell the computer where in its main memory to find the data, it just points to one of the CPU's own internal registers.

c) Register Indirect Mode

Imagine you're trying to find a specific book in a huge library. Instead of the librarian telling you the book's exact shelf number directly, they give you a slip of paper that has the shelf number written on it. You then use that slip of paper to find the shelf, and then the book is on that shelf.

In this computer 'mode' (a way the computer finds information):

- The instruction doesn't give you the actual piece of information (the 'operand') you need.
- Instead, the instruction tells you which 'register' to look at.
- A 'register' is like a small, super-fast storage spot inside the computer's main processing unit (the CPU).

What's stored in that register isn't the data itself, but the memory address* where the data is located.

So, the computer first looks inside the specified register to find the address, and then* it goes to that memory address to get the actual data it needs for the instruction.

Advantages

Imagine you're trying to find a specific book in a huge library. If the book is right next to your desk, you can grab it super fast! If it's in a far-off aisle, it takes much longer.

- **Faster memory access to the operand(s):** This means the computer can get the data (called an **operand**) it needs to work with much quicker. In some cases, like with **Register Mode** (where the data is in a quick-access spot inside the computer's brain, the CPU), it's like the book is already on your desk.

Think about instructions like a to-do list. If each item on the list is short and to the point, you can read and understand the whole list faster. If each item is very long and complicated, it takes more time to go through it.

- **Shorter instructions and faster instruction fetch:** This means the commands the computer receives are more compact. Because they're shorter, the computer can grab them (**fetch** them) from where they're stored more quickly. This is similar to how shorter to-do list items make it faster to get through your tasks.

Disadvantages

While using multiple special scratchpads called 'registers' can make a computer run faster, it can also make the instructions it needs to follow more complicated.

Another limitation is that the 'address space' is very limited. Think of the address space like the number of mailboxes you have available. If you have only a few mailboxes, you can only store a limited number of letters (data or instructions) and you can't easily send more.

- Having a small address space means the computer can't access or store as much information.
- More registers can speed things up but make the instructions harder to understand.

d) Auto Increment/Decrement Mode

Imagine you're reading a list of items, and after you check off an item, you automatically move to the next one on the list, or if you're going backwards, you move to the previous one. Auto Increment/Decrement Mode is a bit like that for the computer.

This mode is a way for instructions to find the data they need. It's an 'addressing mode', which is just a fancy term for how the computer figures out where the data is.

Here's how it works:

- **Auto Increment:** After the computer uses the data found at a specific location, the location's address is automatically increased by one. It's like ticking off an item on a shopping list and then automatically pointing to the next item.
- **Auto Decrement:** Before the computer uses the data, the location's address is automatically decreased by one. This is like getting ready to grab an item from the end of a shelf and moving your hand to the item just before it.

So, the register (a small, super-fast storage spot inside the computer's processor) either goes up or down by one after or before its value is used to find data.

e) `Direct Addressing Mode

Imagine you're looking for a specific book in a library, and the library catalog directly tells you the exact shelf number where the book is located. You don't need to go through multiple steps or look up another reference to find it.

In computer terms, this is like **Direct Addressing Mode**. The **effective address** (which is simply the exact location in the computer's memory where the data, or 'operand', is stored) is given to you directly within the instruction itself.

For example, if an instruction says `ADD R1, 4000`, the number `4000` is the effective address. The computer knows immediately to go to memory location 4000 to find the data it needs to add to register R1.

This means:

- The computer only needs to look at memory once to find the data.
- It doesn't need to do any extra work or calculations to figure out where the data is. It's already known.

g) Displacement Addressing Mode

Imagine you're trying to find a specific book in a very large library. You have a main catalog (the 'Address part of the instruction') that tells you the general section where the book is located. However, within that section, there might be many shelves, and you have a helper (an 'indexed register') who knows how many shelves down from the start of that section your book is. You add the general section number from the catalog and the number of shelves your helper counted to find the exact spot of your book.

In computer terms:

- **Displacement Addressing Mode** is a way for an instruction to figure out exactly where a piece of data (an 'operand') is stored in the computer's memory.
- It works by taking two numbers and adding them together:
- The first number comes from the 'Address part of the instruction'. Think of this as the main location given by the instruction itself.
- The second number comes from a special storage spot within the computer's processing unit, called an **indexed register**. This register holds an extra offset or 'displacement' value.
- When these two numbers are added, the result is the 'effective address'. This 'effective address' is the final, precise location in memory where the computer can find the data it needs to work with.

f) Indirect Addressing Mode

Imagine you're trying to find a hidden treasure. Instead of being told the treasure's exact location, you're given a map that tells you where to find another map. You have to go to that second location, find the next map, and *then* that map will finally tell you where the treasure is buried. This is like Indirect Addressing Mode.

In computer terms:

- **Instruction:** This is the command the computer receives.
- **Address Field:** This part of the instruction normally tells the computer exactly where to find the data (called an **operand**) it needs to work with. An operand is the piece of data that an instruction acts upon.

Indirect Addressing Mode: *In this mode, the address field of the instruction doesn't point directly to the data. Instead, it points to a location in the computer's memory where the* actual* address of the data is stored.

So, the computer first looks at the address given in the instruction. Then, it goes to that memory location to find the *real* address of the data it needs. Finally, it goes to that real address to get the operand.

This method requires an extra step of looking up the address in memory, which means it takes longer to find the data and execute the instruction compared to other addressing modes. This extra memory lookup is what slows down the execution.

h) Relative Addressing Mode

Imagine you're following a treasure map, but instead of a fixed 'X marks the spot,' the instructions tell you to take a certain number of steps *from where you are right now*.

Relative Addressing Mode is like that. It's a clever way for the computer to find the data it needs by using its current location as a starting point.

Here's how it works:

Program Counter (PC): *This is like your current position on the treasure map. It's a special internal counter in the computer that always keeps track of the* next instruction* to be executed.

- **Instruction Address Part:** This is the 'number of steps' given in the instruction. It tells the computer how far to move from its current position.
- **Effective Address (EA):** This is the final destination – the actual location in memory where the data is found. It's calculated by adding the 'number of steps' (from the instruction) to your 'current position' (the PC).

Think of it as: **Your Current Position + Number of Steps = Final Destination**

This mode is actually a variation of Displacement Addressing Mode, meaning it uses a starting point and adds an offset to find the target.

In simple terms, the computer takes the address stored in the Program Counter (PC) and adds it to the address part of the instruction to figure out the exact spot (Effective Address) where the data is located.

j) Stack Addressing Mode

Imagine a stack of plates. When you want to do something with the plates, you usually interact with the one right at the very top. The Stack Addressing Mode works in a similar way for a computer's instructions.

In this mode, the data (called an **operand**, which is the item an instruction acts upon) we need is located at the top of a special structure called a **stack**. A stack is like that pile of plates – you can only easily access the topmost one.

For example, when a computer sees an instruction like 'ADD', it knows to perform a specific action:

- It takes the top two items from the stack (like taking the top two plates).
- It adds them together.
- Then, it places the result back onto the top of the stack (like putting the combined item back as a new plate on top).

This means the data isn't directly given in the instruction or found at a specific memory address, but is assumed to be waiting at the top of this 'stack' structure.

i) Base Register Addressing Mode

Imagine you have a large filing cabinet where each drawer (a memory location) has a number. Base Register Addressing Mode is like having a special note that tells you which drawer to start looking in (the 'base address'), and then you also have a separate list of specific item numbers (the 'displacement') to find within that starting drawer.

This mode is a type of Displacement Addressing Mode. Displacement addressing is a way for instructions to find data by combining a starting address with an offset. In this specific version:

- **EA** stands for Effective Address. This is the final, exact location in the computer's memory where the data is found.
- **A** is the displacement. Think of this as a specific number or offset from a starting point.
- **R** holds a pointer to the base address. This is like a special marker or register that points to the beginning of a section of memory.

So, the computer calculates the EA by taking the base address (pointed to by R) and adding the displacement (A) to it. It's like saying, 'Start at drawer number 5 (R), and then look for item number 3 (A) within that drawer.'

a) BCD Code or 8421 Code

Imagine you have a special way of writing down numbers using only four special coins. These coins have values, like a coin worth 8, another worth 4, another worth 2, and a final one worth 1. By choosing to use or not use each coin, you can make different sums to represent numbers.

This special way of writing numbers is called BCD, which stands for 'Binary-Coded Decimal.' It's a system that helps computers understand our everyday decimal numbers (0-9) by translating them into a binary (0s and 1s) format. Each decimal digit is represented by a group of four binary bits.

This system is also known as the '8-4-2-1 code' because those numbers (8, 4, 2, and 1) are the 'weights' or values assigned to each of the four binary bits. The 'Least Significant Bit' (LSB), which is the rightmost bit, has a weight of 1. Moving left, the next bit has a weight of 2, then 4, and the 'Most Significant Bit' (MSB), the leftmost bit, has a weight of 8.

Because these weights are used, BCD is called a 'weighted code,' which means you can perform arithmetic operations directly on these coded numbers. For example, to find the decimal value of the binary code 0101:

- Take the first bit (0) and multiply it by its weight (8): 0 * 8 = 0
- Take the second bit (1) and multiply it by its weight (4): 1 * 4 = 4
- Take the third bit (0) and multiply it by its weight (2): 0 * 2 = 0
- Take the fourth bit (1) and multiply it by its weight (1): 1 * 1 = 1
- Add these results together: 0 + 4 + 0 + 1 = 5. So, 0101 in BCD represents the decimal digit 5.

Using four bits, you can represent numbers from 0 (0000) up to 15 (1111). However, BCD is specifically designed to represent decimal digits, which only go up to 9. This means that some combinations of four bits are not used in BCD:

- 1010 (decimal 10)
- 1011 (decimal 11)
- 1100 (decimal 12)
- 1101 (decimal 13)
- 1110 (decimal 14)

- 1111 (decimal 15)

These six combinations are called 'forbidden codes' or the 'forbidden group' in BCD because they don't correspond to any single decimal digit.

When you use BCD to represent a decimal number, it often takes more bits than if you were to convert the entire decimal number to its straight binary equivalent. For instance, the decimal number 35 is represented as 0011 0101 in BCD, which is 8 bits. In straight binary, 35 is 100011, which is only 6 bits.

Despite using more bits, BCD is very convenient for getting information into and out of digital systems, making it easier for us to interact with computers.

3.0 CODES AND THEIR CONVERSIONS

We've seen how computers use binary (0s and 1s) for everything. But sometimes, it's easier for us humans to work with other ways of representing information, especially when dealing with numbers and symbols. This section is all about those different ways of representing information, called 'codes', and how we can switch between them.

We've already touched on BCD (Binary-Coded Decimal). Think of it like using a special, slightly more efficient way to write down numbers that are usually in our everyday decimal system. Instead of using a full binary number for each decimal digit, BCD uses a fixed 4-bit binary code for each digit from 0 to 9. For example, the decimal digit '5' is represented as '0101' in BCD, and the decimal digit '3' is '0011'.

However, BCD has some 'forbidden' codes. Because we're only representing 10 digits (0-9), some 4-bit binary combinations won't be used. For instance, '1100' (which is 12 in regular binary) isn't a valid BCD code for any single decimal digit.

This section is essentially about understanding these different codes and how to convert numbers from one code to another. This is important because computers internally work with binary, but we often need to input or output information in formats that are more understandable to humans, like decimal numbers or specific character codes.

Introductions

Imagine you have different ways to write down information, like using letters, numbers, or even special symbols. In the world of computers, which primarily understand only 'on' and 'off' (represented by 1s and 0s), we need ways to translate all sorts of information into this binary language. This process is called 'coding'.

Computers use these codes for many reasons, like when you type information in, when the computer does calculations, or even to make sure the information hasn't been messed up.

Basically, anything a computer needs to understand – whether it's a letter, a number, or a symbol like '@' – has to be turned into a unique pattern of 1s and 0s. This is how digital circuits, which work with binary signals, can process information that comes in different forms.

Sometimes, within the same computer, different types of codes might be used for different jobs. If you need to switch information from one code to another, you'll use special circuits designed for this 'code conversion'.

There are different categories of codes used in digital systems:

- **Weighted Binary Codes:** Think of these like counting with special coins, where each coin has a specific value (a 'weight'). When you count, you add up the values of the coins you're using. In these codes, each binary digit (0 or 1) is multiplied by its 'weight', and then you add them all up to get the final decimal number.
- **Non-weighted Codes:** These are like counting systems where the position of a digit doesn't necessarily have a fixed, predictable value associated with it.
- **Error-detection Codes:** Imagine sending a postcard and having a special check digit that helps the receiver know if any part of your message got smudged during delivery. These codes add extra bits to the information to help detect if any errors occurred during transmission or storage.
- **Error-correcting Codes:** These are even smarter! They not only detect errors but can also fix them automatically, like a magic pen that can fix smudges on your postcard without you doing anything.
- **Alphanumeric Codes:** These codes are used to represent not just numbers but also letters (alphabetic) and special characters (like !, ?, $) all in binary form.

b) 84-2-1 Code

We've seen how computers use binary (0s and 1s) to represent numbers. Now, let's look at a specific way to use binary to represent our everyday decimal numbers, but with a twist!

Imagine you have a special set of weights for counting, but instead of all positive weights like 1, 10, 100, some of them are negative. The 84-2-1 code is like that. It assigns weights of 8, 4, -2, and -1 to the binary digits (bits) in a specific order to represent decimal digits.

Here's how it works:

- Each group of four bits represents one decimal digit.
- The first bit has a weight of 8.
- The second bit has a weight of 4.
- The third bit has a weight of -2.
- The fourth bit has a weight of -1.

To find the decimal value of a 4-bit combination, you multiply each bit by its corresponding weight and add them up.

For example, the binary combination 0101 represents the decimal digit 3. Here's the calculation:

(0 *8)* + *(1* 4) + (0 *-2)* + *(1* -1) = 0 + 4 + 0 - 1 = 3.

A cool feature of this 84-2-1 code is that it's 'self-complementary'. This means if you flip all the 0s to 1s and 1s to 0s in a code for a decimal number, you get the code for its '9's complement'. The 9's complement of a number is what you need to add to it to get 9 (e.g., the 9's complement of 3 is 6, because 3 + 6 = 9).

Let's see this with the example of decimal 3 (0101):

- If we flip the bits, we get 1010.
- Now, let's calculate the decimal value of 1010 using the 84-2-1 weights:

(1 *8) + (0* 4) + (1 *-2) + (0* -1) = 8 + 0 - 2 + 0 = 6.

- And indeed, 6 is the 9's complement of 3!

Example 3.2

We've seen how numbers can be written in different ways, like using our everyday decimal system or the binary system computers understand. Now, let's look at another way to represent numbers called BCD (Binary-Coded Decimal).

Think of BCD like assigning a unique 4-digit binary code to each individual decimal digit (0 through 9). It's a bit like having a special secret codebook where each number from 0 to 9 has its own specific 4-bit pattern.

In this example, we want to convert the decimal number 69.27 into its BCD equivalent.

Here's how it works:

1. **Take the decimal number:** We start with 69.27.
2. **Convert each decimal digit to its BCD code:**
   - The digit '6' in decimal becomes '0110' in BCD.
   - The digit '9' in decimal becomes '1001' in BCD.
   - The digit '2' in decimal becomes '0010' in BCD.
   - The digit '7' in decimal becomes '0111' in BCD.
3. **Combine the BCD codes:** We put these BCD codes together, keeping the decimal point in its place.

So, the decimal number 69.27 is represented as (0110 1001 . 0010 0111) in BCD.

Example 3.1

Let's look at an example to see how we can represent a regular decimal number using a special binary code called BCD (Binary-Coded Decimal).

Imagine you have a number, like 589. We want to write this using BCD.

Here's how we do it:

- We take each digit of the decimal number (5, 8, and 9) separately.
- For each digit, we find its unique 4-bit BCD code. We saw before that BCD uses specific 4-bit patterns for each decimal digit.
- The decimal digit '5' is represented as '0101' in BCD.
- The decimal digit '8' is represented as '1000' in BCD.

- The decimal digit '9' is represented as '1001' in BCD.
- Then, we just put these BCD codes together in order.

So, the decimal number 589 becomes the BCD code '010110001001'.

Think of it like translating a word into a secret code, where each letter has its own symbol. Here, each decimal digit has its own 4-bit binary symbol.

3.1.2 Nonweighted Codes

Imagine you have different ways to assign a number to something, like assigning a score to a student's test. Some scoring systems give more points for harder questions (that's like a 'weighted' system), while others just assign a score based on the overall performance without a specific value for each individual part.

Nonweighted codes are like that second scoring system. In these codes, each position in the binary number doesn't have a fixed, assigned value that you can multiply by.

Instead, the binary pattern itself represents the information without relying on positional values.

Examples of nonweighted codes include:

- **Excess-3 codes**: These are codes where each decimal digit is represented by a 4-bit binary number that is 3 more than its standard binary representation.
- **Gray codes**: These are codes where consecutive values differ by only one bit. This is super useful when you have something that's changing and you want to avoid errors during the transition.

c) 2421 Code

Imagine you have a special way of writing down numbers using only 0s and 1s, but instead of each position having a value of 1, 2, 4, 8 (like we saw before), this new system uses different values for each position.

The 2421 code is one such system. It's a 'weighted code', meaning each of the four binary digits (bits) used to represent a decimal digit has a specific value, or 'weight', assigned to it. These weights are 2, 4, 2, and 1, from left to right.

For the decimal digits 0 through 4, the 2421 code uses the exact same binary patterns as the standard BCD (Binary-Coded Decimal) code we've encountered. However, for digits 5 through 9, the patterns are different.

Let's look at an example: The binary pattern 0100 in the 2421 code represents the decimal digit 4. This makes sense because the weights are 2, 4, 2, 1, so 0*2* + *1*4 + 0*2* + *0*1 = 4.

Now, consider the binary pattern 1101. To find its decimal value in the 2421 code, we multiply each bit by its corresponding weight and add them up: (1 *2)* + *(1* 4) + (0 *2)* + *(1* 1) = 2 + 4 + 0 + 1 = 7. So, 1101 represents the decimal digit 7 in this code.

This 2421 code has a neat property called being 'self-complementary'. This means if you take a binary pattern for a decimal digit and flip all the 0s to 1s and all the 1s to 0s, you get the binary pattern for the decimal digit that, when added to the original digit, equals 9. For example, if 0100 is 4, flipping it gives 1011. If we calculate 1011 using the 2421 weights: (1*2*) + (*0*4) + (1*2*) + (*1*1) = 2 + 0 + 2 + 1 = 5. And indeed, 4 + 5 = 9.

a) Excess-3 Code

Imagine you have a special way of writing down numbers, like a secret code. The Excess-3 code is one such code that some older computers used to represent the digits from 0 to 9.

How it works:

1. **Start with the decimal digit:** Take the number you want to represent (like 0, 1, 2, up to 9).

2. **Add 3:** Add the number 3 to that decimal digit.

3. **Convert to binary:** Then, convert that new number (the original digit plus 3) into its 4-bit binary form. This 4-bit binary pattern is the Excess-3 code for your original digit.

For example:

- To find the Excess-3 code for the decimal digit '2':
- Add 3: 2 + 3 = 5
- Convert 5 to 4-bit binary: 0101. So, 0101 is the Excess-3 code for 2.
- To find the Excess-3 code for the decimal digit '9':
- Add 3: 9 + 3 = 12
- Convert 12 to 4-bit binary: 1100. So, 1100 is the Excess-3 code for 9.

This code is also 'nonweighted', meaning the bits don't have fixed values like in some other binary systems. However, it has a neat trick: it's 'self-complementary'. This means if you flip all the 1s to 0s and all the 0s to 1s in an Excess-3 code, you get the Excess-3 code for the '9's complement of the original decimal digit. This property is really helpful for doing subtraction in computers because it can be done using addition. The maximum value that can be represented with a 4-bit binary code is 1100 (which is 12 in decimal), and since we only need to represent digits up to 9, adding 3 ensures we have enough unique patterns.

Example 3.4

Let's see how to convert a decimal number into its Excess-3 code. Remember, Excess-3 code is a way to represent decimal digits using binary, where you add 3 to the decimal digit's value and then convert that sum into a 4-bit binary pattern.

We want to convert the decimal number (58.43) into Excess-3 code.

Here's how we do it, step-by-step:

1. **Convert each decimal digit:** We take each digit of '58.43' and add 3 to it.

- 5 + 3 = 8
- 8 + 3 = 11
- 4 + 3 = 7
- 3 + 3 = 6

2. **Convert the sums to 4-bit binary:** Now, we take each of these results and convert them into their 4-bit binary representation.

- 8 becomes 1000
- 11 becomes 1011
- 7 becomes 0111
- 6 becomes 0110

3. **Combine the binary parts:** We put these 4-bit binary patterns together, keeping the decimal point in its place.

So, the Excess-3 code for (58.43) in decimal is (1000 1011.0111 0110).

This shows that the Excess-3 code for (58.43)■■ is (1000 1011.0111 0110).

Example 3.3

Let's walk through an example of converting a decimal number into its Excess-3 code. We already learned that Excess-3 code is a way to represent decimal digits using binary, where you first add 3 to the decimal digit and then convert that sum into a 4-bit binary pattern.

Imagine you have a secret code where each number needs a little 'boost' before you write it down in a special binary format. For Excess-3, that boost is always adding 3.

Problem: Convert the decimal number (367)■■ into its Excess-3 code.

Steps:

1. **Break down the decimal number:** Our number is 367, which means we have the digits 3, 6, and 7.

2. **Add 3 to each digit:**

- For the digit '3': 3 + 3 = 6
- For the digit '6': 6 + 3 = 9
- For the digit '7': 7 + 3 = 10

3. **Convert each sum to its 4-bit binary equivalent:**

- The sum '6' in 4-bit binary is 0110.
- The sum '9' in 4-bit binary is 1001.
- The sum '10' in 4-bit binary is 1010.

4. **Combine the 4-bit binary patterns:** Put these 4-bit binary representations together in the same order as the original decimal digits.

So, the Excess-3 code for (367)■■ is 0110 1001 1010.

This shows how each decimal digit is transformed into a specific 4-bit binary sequence following the Excess-3 rule.

Example 3.5. Convert (101011)2 Solution

This example shows us how to convert a binary number (a sequence of 0s and 1s that computers understand) into something called a Gray code. Think of Gray code like a special secret code where changing one number in the original binary sequence only changes one digit in the secret code, making it useful for preventing errors.

Here's how we convert the binary number 101011 to its Gray code:

- **Step 1: The First Digit is Easy!** The very first digit (the leftmost one, also called the Most Significant Bit or MSB) of the Gray code is exactly the same as the first digit of the original binary number. So, our Gray code starts with a '1'.
- **Step 2: Mixing the First Two.** Now, we look at the first two digits of the binary number (1 and 0). We perform a special operation called an 'ex-OR' (exclusive OR) between them. If the two digits are different, the result is 1. If they are the same, the result is 0. In this case, 1 ex-OR 0 is 1. This '1' becomes the second digit of our Gray code.
- **Step 3: Mixing the Next Pair.** Next, we take the second and third digits of the binary number (0 and 1) and perform the ex-OR operation. 0 ex-OR 1 is 1. This '1' becomes the third digit of our Gray code.
- **Step 4: Keep Going!** We continue this pattern. We take the third and fourth digits of the binary number (1 and 0) and do an ex-OR. 1 ex-OR 0 is 1. This '1' becomes the fourth digit of our Gray code.
- **Step 5: The Last Mix.** Finally, we take the fourth and fifth digits of the binary number (0 and 1) and perform the ex-OR. 0 ex-OR 1 is 1. This '1' becomes the fifth digit of our Gray code.
- **Step 6: The Final Digit.** We take the fifth and sixth digits of the binary number (1 and 1) and perform the ex-OR. 1 ex-OR 1 is 0. This '0' becomes the sixth and final digit of our Gray code.

Putting it all together, the Gray code for the binary number 101011 is 111110.

Binary to Gray Code Conversion Steps

Let's learn how to turn a regular binary number into a special 'Gray code'. Think of it like translating a message where each letter's meaning depends on the letter before it.

Here's how it works, step-by-step:

1. **The First Bit is Easy:** The very first binary digit (the Most Significant Bit, or MSB – the leftmost one) in your original binary number is exactly the same as the first digit in your new Gray code. No changes here!

2. **The Second Bit's Secret:** To find the second digit of the Gray code, you look at the first two digits of your original binary number. You perform an 'Exclusive OR' (Ex-OR) operation on them.

What's Ex-OR? *It's a simple logic rule: if the two binary bits you're comparing are the* same *(both 0 or both 1), the result is 0. If they are* different* (one is 0 and the other is 1), the result is 1.

3. **Continuing the Pattern:** For every subsequent bit in the Gray code, you take the *previous* binary bit and the *current* binary bit, and perform the same Ex-OR operation. You keep doing this until you've converted all the bits from the original binary number.

Essentially, you're comparing adjacent bits in the binary number and using their difference (or sameness) to create the new Gray code bits.

b) Gray Code

Imagine you're trying to track the position of a spinning wheel, like the one at a carnival game. You want to know its exact position, but you don't want a bunch of numbers changing all at once if the wheel moves just a tiny bit. Gray code is like a special way of labeling the positions on that wheel so that as it moves from one spot to the next, only one little part of the label changes at a time.

This type of code is called a 'minimum change code' because when you go from one number to the next, only one bit (a 0 or a 1) flips.

This is super useful in real-world gadgets:

- **Shaft encoders:** These are devices that measure rotational or linear movement. Gray code helps them accurately report the position without errors from slight movements.
- **Analog-to-digital converters:** These devices turn real-world analog signals (like sound waves) into digital numbers the computer can understand. Gray code can be used here to ensure smooth transitions between digital values.

Unlike some other number systems we've talked about, Gray code is **not a weighted code**. This means each bit doesn't have a fixed value based on its position; its purpose is purely to ensure that only one bit changes between consecutive numbers.

Converting Gray Code to Binary

Now that we know about Gray code, which is a special way to represent numbers where each step only changes one bit, let's see how we can turn it back into a regular binary number. Think of it like having a secret code (Gray code) and figuring out the original message (binary).

Here's how we can decode Gray code back into binary:

1. **The first step is easy:** The very first digit (the Most Significant Bit, or MSB) of your binary number is exactly the same as the first digit of your Gray code. It's like the first letter of the secret code is the same as the first letter of the original message.

2. **Finding the next digit:** To get the second digit of your binary number, you compare the first binary digit (which you just found) with the second digit of the Gray code. You use something called an 'Exclusive OR' (Ex-OR) operation.

- If the two digits you're comparing are the same (both 0 or both 1), the result is 0.

- If the two digits are different (one is 0 and the other is 1), the result is 1.

This is like a simple rule to figure out the next letter based on the first letter and the next secret code letter.

3. **Continuing the pattern:** For every digit after that, you do the same thing. Take the binary digit you just figured out and 'Ex-OR' it with the next digit from the Gray code. This process continues for all the remaining digits, working your way down from left to right.

Essentially, you're using the previously calculated binary bit and the current Gray code bit to determine the next binary bit.

Example 3.6. Convert (564)10 to Gray code Solution

We're going to convert the decimal number 564 into its Gray code equivalent.

Think of Gray code like a special way of counting where each new number is only slightly different from the last, like turning a knob one click at a time. This helps prevent errors when things are changing quickly.

Step 1: First, we need to change our decimal number (which is what we use every day) into a binary number (which is what computers understand). The material previously showed us how to do this using division and keeping track of remainders.

The decimal number 564 needs to be converted to binary.

Once we have the binary representation, we'll use that to find the Gray code. (The actual conversion steps from binary to Gray code were explained in a previous section, involving comparing adjacent binary bits with an Exclusive OR operation).

Example 3.7. Convert the Gray code 101101 into a binary number

We're going to convert a Gray code number into its binary equivalent. Think of it like translating a secret code back into a regular language.

Here's how we do it, step-by-step, using the Gray code 101101:

- **Step 1: Find the first bit.** The very first bit (the leftmost one, also called the Most Significant Bit or MSB) of the binary number is exactly the same as the MSB of the Gray code. So, our binary number starts with 1.

Step 2: Find the second bit. *Now, we take the MSB of the binary number we just found (which is* 1*) and perform an 'Exclusive OR' (or 'ex-OR') operation with the* second* bit of the Gray code (which is 0). The result of 1 ex-OR 0 is 1. This 1 becomes the second bit of our binary number. So far, our binary number is 11.

Quick reminder on 'ex-OR':* It's a logic operation where the result is 1 if the two bits are different, and 0 if they are the same.

Step 3: Find the third bit. *We take the* second *bit of the binary number we've built so far (which is* 1*) and perform an 'ex-OR' with the* third* bit of the Gray code (which is 1). The result of 1 ex-OR 1 is 0. This 0 is

the third bit of our binary number. Our binary number is now `110`.

- **Step 4: Continue the pattern.** We keep repeating this process:

Take the previous *binary bit and 'ex-OR' it with the* next* Gray code bit.

The result is the next* binary bit.

Let's finish our example:

- Fourth bit: Previous binary bit (`0`) ex-OR next Gray bit (`1`) = `1`. Binary is now `1101`.
- Fifth bit: Previous binary bit (`1`) ex-OR next Gray bit (`0`) = `1`. Binary is now `11011`.
- Sixth bit: Previous binary bit (`1`) ex-OR next Gray bit (`1`) = `0`. Binary is now `110110`.

So, after converting the Gray code `101101`, the resulting binary number is `110110`.

a) Parity Bit Coding Technique

Imagine you're sending a secret message written on a piece of paper. Sometimes, during delivery, a smudge or a tear can mess up a few letters, making the message confusing. The Parity Bit Coding Technique is like adding an extra, special check mark to your message to help the receiver know if any part of it got smudged.

Here's how it works:

1. **What is a Parity Bit?**
   - A parity bit is an extra bit (a 0 or a 1) that's added to a group of message bits (the actual information).
   - Its job is to make the total count of '1's in the entire group (message bits + parity bit) either always odd or always even.
2. **How it Helps Detect Errors:**
   - When you send a message, it travels through things like wires or radio waves. These can introduce 'noise,' which might flip a 0 to a 1, or a 1 to a 0. This is an 'error' in your message.
   - At the sending end, a special circuit called a **parity generation** circuit calculates the parity bit based on the message bits.
   - The message and its parity bit are sent to the destination.
   - At the receiving end, another circuit, the **parity check** circuit, looks at all the incoming bits (message + parity bit).
   - It checks if the total number of 1s matches the expected parity (odd or even) that was agreed upon when sending.
   - If the parity doesn't match what's expected, it means an error has likely occurred. The system can then indicate that an error was detected.
3. **Odd vs. Even Parity:**
   - **Odd Parity:** The parity bit is chosen so that the total number of 1s (including the parity bit) is an odd number. For example, if you have a message '0001', it has one '1'. To make it odd, the parity bit would be '0', making the total '00010' (still one '1'). If the message was '0011', it has two '1's. To make it odd, the parity bit would be '1', making the total '00111' (three '1's).

• **Even Parity:** The parity bit is chosen so that the total number of 1s (including the parity bit) is an even number. For example, if you have a message '0001' (one '1'), the parity bit would be '1' to make the total '00011' (two '1's).

4. **Where to Put the Parity Bit:**

• The parity bit can be added either at the beginning (the **MSB**, or Most Significant Bit side) or the end (the **LSB**, or Least Significant Bit side) of the message bits. Sometimes, if you add it in a different spot, you might need other coding schemes like 'Check Sums' (which are like a more advanced way to check for errors).

5. **Limitations:**

*This method is good at detecting errors, but it **cannot correct** them. It just tells you* if* something went wrong.

• It can detect if there's an odd number of errors (like one, three, five, etc.).

• However, if there's an even number of errors (like two or four bits flipped), the parity might still look correct, and the error would go undetected. It's like if two smudges accidentally cancel each other out and the message still looks okay.

3.1.3 Error-detection Codes

Imagine you're sending a secret message written on tiny slips of paper, and sometimes a few letters get smudged or changed when you pass them along. Error-detection codes are like adding a special 'check' to your message so the person receiving it can tell if something went wrong.

Think of it like having a friend count the number of apples you give them. If you give them 5 apples, and they count 5, they know everything is okay. But if they count 4, they know something's missing!

Parity Bit:

This is a simple way to check for errors. We add an extra bit (a 0 or a 1) to a group of data bits. This extra bit is chosen so that the total number of 1s in the whole group (including the parity bit) is either always odd or always even. It's like agreeing beforehand to always have an odd number of red marbles in a bag.

• **How it works:** If the receiver counts the 1s and the total doesn't match the agreed-upon rule (e.g., it should be even, but they count an odd number of 1s), they know an error occurred.

***Limitation:** This simple parity check can only detect* if *an error happened, but it can't tell you* where* the error is or how to fix it. It's like knowing a letter is smudged, but not knowing which letter it is.

b) Check Sums

Imagine you're sending a list of numbers to a friend. To make sure your friend gets the exact same list, you could add up all the numbers yourself and send that total along with the list. When your friend receives the list, they add up all the numbers they received and compare their total to the total you sent. If they match, it's very likely the list arrived correctly!

Check sums work similarly for computers.

Here's how it works:

• **Adding it up:** When a computer sends information (like a "word," which is just a chunk of data), it adds up all the parts of that word. This running total is kept track of.

• **Continuing the sum:** As more "words" of data are sent, each new word is added to the total that was already kept. This process continues for all the data being sent.

• **The final total (Check Sum):** Once all the data has been sent, the very last running total is called the "Check Sum." This Check Sum is then sent along with the original data.

• **Checking at the other end:** The computer receiving the data does the exact same addition process with all the data it received. It calculates its own final total.

• **The moment of truth:** The receiving computer then compares its calculated total with the Check Sum that was sent. If the two totals are equal, it means the data was likely transmitted without any errors.

This method is more robust than simple parity bits because it can detect more types of errors, especially when multiple bits might have changed during transmission.

a) Hamming Code

Imagine you're sending a secret message, but sometimes a letter might get smudged or changed by accident. The Hamming Code is like a clever way to add a few extra 'check letters' to your message so that if a mistake happens, you can not only spot that a mistake occurred but also figure out exactly which letter was smudged and fix it!

Here's how it works:

• **What is a Hamming Code?** It's a special method developed by R. W. Hamming. It adds extra bits, called **parity bits**, to your data in a very organized way. These parity bits act like little detectives that can find and fix errors.

• **Hamming Distance:** This is a way to measure how different two pieces of coded information are. If you have two coded messages, the Hamming distance is simply the count of the positions where their bits are different. For example, if one message is `0101` and another is `0011`, they differ in two positions (the second and fourth bits), so their Hamming distance is 2.

• **Minimum Hamming Distance (d_min):** This is the smallest Hamming distance you can find between any two possible coded messages in a set. A larger minimum Hamming distance means the code is better at detecting and correcting errors.

• **Detecting and Correcting Errors:**

• To just **detect** a single error, your code needs a minimum Hamming distance of at least 2 (d_min >= 2). This means if a single bit flips, the resulting code will be different from the original, and you can tell something's wrong.

• To **correct** a single error, your code needs a minimum Hamming distance of at least 3 (d*min >= 3). This is because if a single bit flips, you can tell which original code it was supposed to be by comparing it to the possibilities. The formula for how many errors you can correct is roughly (d*min - 1) / 2. So, with d_min = 3, you can correct (3-1)/2 = 1 error.

• **How Parity Bits Work (Simplified):**

• Parity bits are placed at specific positions within the code, usually corresponding to powers of two (like positions 1, 2, 4, 8, etc.).

• Each parity bit is responsible for checking a specific group of data bits. The groups are determined by the binary representation of the parity bit's position.

• For example, a parity bit at position 2 (binary `010`) checks all bit positions that have a `1` in their second bit from the right (positions 2, 6, 10, etc.). A parity bit at position 4 (binary `100`) checks all bit positions that have a `1` in their leftmost bit (positions 4, 5, 6, 7, etc.).

• These parity bits are usually set to ensure **even parity**, meaning the total number of `1`s in the group they check (including the parity bit itself) should be an even number.

• **Finding an Error:**

• When you receive a Hamming code, you recalculate the parity bits based on the received data bits. If a parity check fails (meaning you don't have an even number of 1s where you should), it signals an error.

• The combination of failed parity checks (often called a **syndrome**) acts like an error locator. If the syndrome bits are `000`, it means there's no error. If the syndrome bits have a non-zero value, like `110`, the binary value `110` (which is 6 in decimal) tells you that the 6th bit is the one that's wrong.

• **Correcting an Error:** Once you know which bit is in error (e.g., bit 6), you simply flip it (change a `0` to a `1` or a `1` to a `0`) to correct the code.

• **Example:** Imagine you receive a 7-bit Hamming code `0110110`. You'd perform parity checks. Let's say the checks indicate that bit 2 is in error (perhaps the syndrome is `010`, which is 2 in binary). To correct it, you flip the 2nd bit of `0110110` from `1` to `0`, resulting in the corrected code `0010110`.

3.1.4 Error-correcting Codes

Imagine you're sending a secret message written on a postcard. Sometimes, the ink might smudge, or a letter might get smudged, making it hard to read. Error-correcting codes are like adding a special secret handshake or a clever way to re-arrange the letters so that even if a little bit gets smudged, the person receiving the postcard can figure out what the original message was.

Error-correcting codes are advanced versions of error-detection codes. While error-detection codes just tell you IF something went wrong, error-correction codes can actually FIX the mistake.

• **Error-correction codes** are like a super-smart proofreader that not only spots mistakes in data but also knows how to correct them.

• They build upon the idea of adding extra bits (like the parity bit we saw before) to the original data.

• These extra bits are carefully calculated so that if a single bit (a 0 or a 1) gets flipped during transmission (meaning it accidentally turns into the other value), the code can pinpoint exactly which bit is wrong and flip it back to its correct value.

• This makes them essential for systems where data integrity is extremely important, like in long-term data storage or reliable communication over noisy channels.

3.1.5 Alphanumeric Codes

We've talked about how computers use binary (0s and 1s) to represent numbers. But what about letters and other symbols we use every day, like 'A', 'B', '#', or '%'?

To handle these, computers need special 'alphanumeric codes'. Think of it like a secret language or a special keyboard layout that lets the computer understand not just numbers, but also letters and symbols.

Here's what makes up an alphanumeric code:

- It includes the ten decimal digits (0-9).
- It includes the 26 letters of the alphabet, and importantly, it needs to handle both uppercase (A, B, C) and lowercase (a, b, c) versions.
- It also needs to represent a bunch of special symbols you find on a keyboard, like '#', '/', '&', '%', and others.

Because there are so many different characters (letters, numbers, symbols) to represent, we need a certain number of binary digits, called 'bits', to create unique codes for each one. The text tells us that we need at least 6 bits. Why 6 bits? Well, with 5 bits, we can only create 32 different combinations (2 to the power of 5), which isn't enough for all the letters, numbers, and symbols. But with 6 bits, we can create 64 different combinations (2 to the power of 6), which is more than enough!

3.1.7 EBCDIC

Imagine you have a special secret code that only certain types of machines understand, especially older or larger computers made by a company called IBM. This code is how they talk about letters, numbers, and symbols.

EBCDIC is one of these special codes. Its full name is "Extended Binary Coded Decimal Interchange Code." It's a way to represent letters and numbers using binary (those 1s and 0s we've talked about).

Here's how it works:

- **It's an 8-bit code:** This means it uses 8 binary digits (bits) to represent each character. Think of it like having 8 slots to fill with 1s or 0s to make a unique pattern for each letter or number.
- **It uses a ninth bit for checking:** On top of the 8 bits for the character itself, it adds a ninth bit. This extra bit is called a **parity bit**, and it's used to help check if the data was transmitted correctly, kind of like a quick count to see if everything looks right.
- **It's different from ASCII:** You might remember we talked about other codes before. EBCDIC groups its characters differently than a code called ASCII, which is another common way to represent text.

3.1.6 ASCII

Imagine you have a special secret codebook that allows you to send messages to your computer, and this codebook uses a specific set of symbols. ASCII (pronounced "as-kee") is like a widely adopted version of this codebook, especially popular for most personal computers.

Here's how it works:

• **The Basic Code:** At its heart, ASCII uses a 7-bit code. This means each character (like a letter, number, or symbol) is represented by a unique pattern of seven 0s and 1s.

• **Storing Characters:** When your computer stores a character using ASCII, it typically uses a full byte (which is 8 bits). The seventh bit is used for the ASCII code, and the eighth bit is often left unused or can be used for other purposes.

• **Expanding the Codebook:** The extra, unused bit in the byte can be cleverly used to expand the ASCII codebook. This allows for an additional 128 characters to be represented, giving you more options for symbols and characters beyond the basic set.

4.0 BOOLEAN ALGEBRA

Imagine you have a special kind of math that deals with only two possibilities: 'yes' or 'no', 'on' or 'off', or in computer terms, '1' or '0'. This is called Boolean algebra.

Boolean algebra is super useful for understanding and building digital systems like computers. It helps us in two main ways:

1. **Understanding How Things Work (Analysis):** It's like having a simple, clear way to describe exactly what a piece of electronic circuitry does. Instead of complex diagrams, we can use this math to explain its function economically.

2. **Building Things (Design):** If you have a specific job you want a circuit to do, Boolean algebra helps you figure out the simplest and most efficient way to build it.

In this system, we use 'logical variables' that can only be either 1 (which we can think of as TRUE) or 0 (which we can think of as FALSE).

The basic building blocks of this math are 'logical operations'. The most common ones are:

• **AND:** This is like saying "both things must be true." If you need milk AND eggs to make pancakes, you need both.

• **OR:** This is like saying "at least one of these things must be true." If you want to go to the park OR the movies, you can choose either one.

• **NOT:** This is like saying "the opposite." If it's NOT raining, it means it's sunny (or at least not raining).

These operations are often written with special symbols: a dot ('.') for AND, a plus sign ('+') for OR, and an overbar (like a small line above a letter) for NOT.

We can also use other operations like NOR, NAND, and XOR, which are combinations of the basic ones.

To see exactly how these operations work with different inputs, we use something called a **truth table**. A truth table is like a chart that lists all the possible combinations of inputs and shows you what the output will be for each combination. It's a clear way to map out the behavior of these logical operations, especially when we have two input variables, but we can extend it for more.

This whole system of Boolean algebra was really brought into focus by Claude Shannon in 1938, who realized it was perfect for designing circuits that used switches. It's named after George Boole, an English

mathematician who developed its core ideas way back in 1854.

4.1 Gates

Imagine you have a set of light switches that control a light bulb. Gates are like special kinds of switches that work together to make decisions based on the input they receive.

At the heart of all digital circuits are these things called **gates**. They are the fundamental building blocks.

A **gate** is an electronic circuit that takes one or more input signals (like the position of our light switches) and produces a single output signal. This output is the result of a simple logical operation performed on the inputs. Think of it like a tiny decision-maker.

The basic types of gates we'll look at are AND, OR, NOT, NAND, NOR, and XOR. Each one has a specific way of making its decision.

We can understand each gate in three ways:

- **Graphic Symbol:** A unique drawing that represents the gate.
- **Algebraic Notation:** A mathematical way to write down what the gate does.
- **Truth Table:** A list showing all possible inputs and the resulting output for each combination.

Let's look at the common ones:

AND Gate:

Analogy: *Imagine a security system that only turns on the alarm if* both* a door sensor AND a window sensor are triggered. If only one is triggered, nothing happens.

Function: *An AND gate's output is '1' (on) only if* all* of its inputs are '1'. If even one input is '0' (off), the output is '0'.

Algebraic Notation: *Often written as A* B, or simply AB.

OR Gate:

Analogy: *Think of a doorbell that rings if* either* person A OR person B presses their button. It doesn't matter who presses it, as long as at least one does.

Function: *An OR gate's output is '1' if* any *of its inputs are '1'. The output is only '0' if* all* inputs are '0'.

- **Algebraic Notation:** Often written as A + B.

NOT Gate:

- **Analogy:** This is like a light switch that reverses the state of a light. If the switch is 'on', the light is 'off', and if the switch is 'off', the light is 'on'.
- **Function:** A NOT gate takes a single input and flips it. If the input is '0', the output is '1'. If the input is '1', the output is '0'.
- **Algebraic Notation:** Often written as A' or ¬A.

NAND Gate:

Analogy: *This is like an AND gate followed by a NOT gate. So, it's like a security system that is* normally *on, but turns off* only if* both the door sensor AND the window sensor are triggered.

Function: *The output is '0' only if* all* inputs are '1'. Otherwise, the output is '1'. It's the opposite of an AND gate.

Algebraic Notation: *Often written as ¬(A* B) or (AB)'.

NOR Gate:

Analogy: *This is like an OR gate followed by a NOT gate. So, it's like a doorbell that is* normally *ringing, but stops ringing* only if both* person A OR person B presses their button.

Function: *The output is '1' only if* all* inputs are '0'. Otherwise, the output is '0'. It's the opposite of an OR gate.

- **Algebraic Notation:** Often written as ¬(A + B) or (A + B)'.

XOR Gate (Exclusive OR):

Analogy: *Imagine a light switch that turns on only if* exactly one* of two buttons is pressed. If both are pressed, or neither is pressed, the light stays off.

Function: *The output is '1' if the inputs are* different* (one is '0' and the other is '1'). If the inputs are the same (both '0' or both '1'), the output is '0'.

- **Algebraic Notation:** Often written as A ⊕ B.